Railway Fare and Reservation Calculator

Problem Statement: Calculate railway fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and cancellation penalties.

1. Computational Thinking

1. Decomposition

- Enter passenger details.
- Enter journey distance.
- Select travel class.
- Calculate fare using distance slabs.
- Apply age-based concessions.
- Add reservation fees.
- Calculate cancellation penalties if required.
- Generate the final ticket statement.

2. Pattern Recognition

- Longer distances have higher fares.
- Different travel classes have different rates.
- Senior citizens or eligible children may receive concessions.
- Cancellation charges depend on cancellation rules.

3. Abstraction

Consider only passenger age, distance, travel class, ticket fare, concession, reservation fee, and cancellation status.

4. Algorithm Design

Divide the railway fare calculation into simple steps and calculate the final amount for each passenger.

2. Variables and Data Types

Variable	Data type	Purpose
name	String	Passenger name
age	Integer	Passenger age

distance	Integer	Journey distance
travel_class	String	Selected class
base_fare	Float	Basic ticket fare
concession	Float	Age-based discount
reservation_fee	Float	Reservation charges
penalty	Float	Cancellation charges
total_fare	Float	Final ticket amount
passengers	List	Stores passenger details
fares	List	Stores passenger fares
record	Tuple	Stores passenger information
count	Integer	Number of passengers

3. Operators Used

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
>	Greater than
<	Less than
==	Equal to
and	Logical AND
or	Logical OR

4. Conditional Statements

- if: Checks passenger age.
- elif: Checks distance slabs and travel class.
- else: Handles other conditions.

- if: Checks whether a passenger is eligible for concession.
- if: Checks whether the ticket is cancelled.

5. Loops

- for loop: Calculates fares for multiple passengers.
- while or for loop: Repeats passenger entry.
- for loop: Displays all passenger ticket details.

6. Strings and Pattern Generation

Strings: Used for passenger names, travel class, ticket status, and messages.

Pattern generation: Displays the railway ticket statement.

==========

RAILWAY TICKET

==========

Passenger Name : Ravi

Age : 65

Distance : 300 KM

Travel Class : Sleeper

Base Fare : Rs. 600

Concession : Rs. 60

Reservation : Rs. 40

Total Fare : Rs. 580

==========

7. Lists

Lists store passenger names and their ticket fares.

passengers = ["Ravi", "Rani", "Kiran"]fares = [580, 750, 900]

8. Tuples and Tuple Operations

Tuples store passenger details.

record = ("Ravi", 65, 300, "Sleeper",
580)print(record[0])print(record[1])print(record[2])

Tuple unpacking:

name, age, distance, travel_class, fare = recordprint(name, age, distance, travel_class, fare)

9. Algorithm

1. Start.
2. Input the number of passengers.
3. Create lists to store passenger details and fares.
4. Repeat for each passenger:
    - Enter passenger name and age.
    - Enter journey distance.
    - Select travel class.
    - Calculate base fare using distance slabs.
    - Apply travel-class fare rates.
    - Check age-based concession eligibility.
    - Calculate concession.
    - Add reservation fee.
    - Check cancellation status.
    - If cancelled, calculate cancellation penalty.
    - Calculate final fare.
    - Store passenger details and fare.
5. Display the passenger ticket statement.
6. Repeat until all passengers are completed.
7. Display total railway fare.
8. Stop.

10. Pseudocode

START

INPUT number_of_passengers

CREATE passenger_list

```
CREATE fare_list

FOR each passenger:

    INPUT name
    INPUT age
    INPUT distance
    INPUT travel_class

    IF distance <= 100:
        base_fare = distance * 2

    ELSE IF distance <= 300:
        base_fare = distance * 3

    ELSE:
        base_fare = distance * 4

    IF travel_class == "Sleeper":
        base_fare = base_fare * 1

    ELSE IF travel_class == "AC":
        base_fare = base_fare * 2

    ELSE:
        base_fare = base_fare * 0.5

    IF age >= 60:
        concession = base_fare * 0.10
```

```
    ELSE:
        concession = 0

    reservation_fee = 40

    fare = base_fare - concession + reservation_fee

    INPUT cancellation_status

    IF cancellation_status == "Yes":
        penalty = fare * 0.20
        fare = penalty
    ELSE:
        penalty = 0

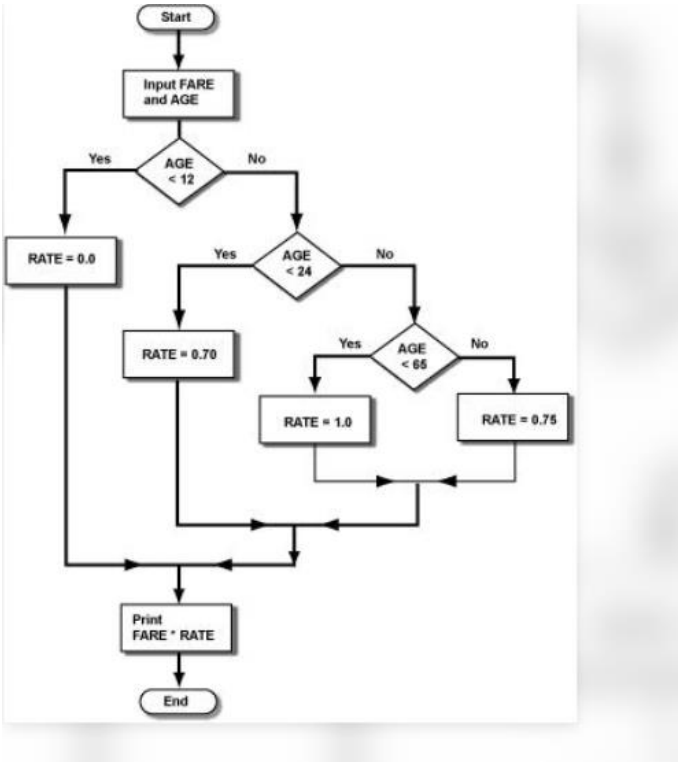
    ADD passenger details TO passenger_list
    ADD fare TO fare_list

    DISPLAY passenger ticket

DISPLAY all passenger details
DISPLAY total fare

STOP
```

11. Flowchart



Simple Text Flowchart

START

|

v

Input Number of Passengers

|

v

Input Passenger Details

(Name, Age, Distance, Class)

|

v

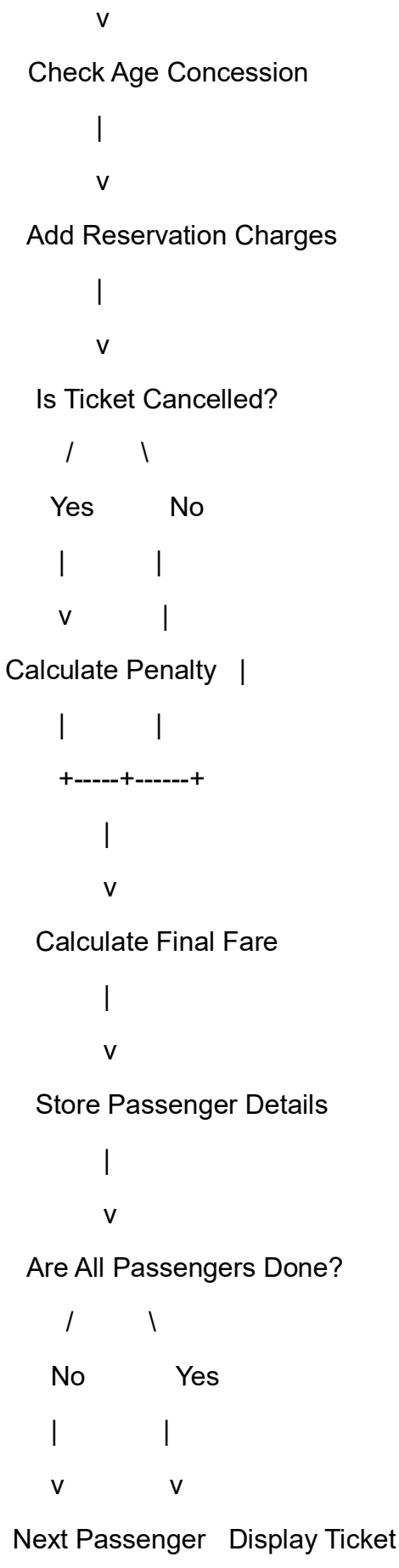
Calculate Distance Fare

|

v

Apply Class Fare Rate

|


v
Check Age Concession
|
v
Add Reservation Charges
|
v
Is Ticket Cancelled?
/ \
Yes No
| |
v |
Calculate Penalty |
| |
+-----+------+
|
v
Calculate Final Fare
|
v
Store Passenger Details
|
v
Are All Passengers Done?
/ \
No Yes
| |
v v
Next Passenger Display Ticket

Details

|

v

Display Total Fare

|

v

STOP

12. Conclusion

The Railway Fare and Reservation Calculator calculates ticket fares for multiple passengers using distance slabs, travel classes, age concessions, reservation fees, and cancellation penalties. It uses computational thinking, algorithms, variables, data types, operators, conditional statements, loops, strings, lists, tuples, and tuple unpacking to generate railway ticket statements.